

PENETRATION TEST REPORT

Target: Placement Portal (Flask Web Application)

Test Type: Gray-box (source code available)

Methodology: OWASP Testing Guide v4

Tester: Sai Nithya Maheswari PK

Date: May 2026

Section 1: Executive Summary

A gray-box penetration test was conducted against the Placement Portal, a Flask-based recruitment web application that I developed as part of an academic project. The assessment covered authentication, access control, API security, session management, and input validation.

Testing identified 7 findings across severity levels: 1 Critical, 2 High, 3 Medium, and 1 Informational. The most significant finding involves an exposed interactive debugger that allows arbitrary code execution on the server when triggered by an authenticated user. Additionally, broken access control logic in the applications API endpoint causes runtime crashes for company-role users.

The application demonstrates strong security architecture in several Areas, CSRF protection successfully blocked all automated form submission attempts, file upload validation correctly enforces MIME type checking beyond extension-based filtering, and student-to-student IDOR on resume access is explicitly prevented.

Immediate remediation is recommended for the Critical and High findings before any production deployment.

Section 2 : Scope and Methodology

Target Application: Placement Portal

URL: <http://127.0.0.1:5000>

Test Environment: Local development (Windows, Python 3.x, SQLite)

Test Type: Gray-box, tester had full source code access

Test Period: May 2026

Methodology: OWASP Testing Guide v4

Phases conducted:

1. Source code review
2. Manual endpoint testing
3. Automated scanning (Nmap, browser-based)
4. Authentication testing
5. Access control testing
6. API security testing

Tools Used:

- Nmap 7.x: port and service enumeration
- Burp Suite Community: request interception
- Browser DevTools: session and response analysis
- PowerShell: automated request scripting
- Flask development server logs: request analysis

In scope:

- All authenticated and unauthenticated routes
- REST API endpoints (/api/*)
- File upload functionality
- Authentication and session management
- Role-based access control (Admin, Company, Student)

Out of scope:

- Third-party libraries and dependencies
- Operating system and network infrastructure
- Physical security

Section 3: Findings

VULN-001 Broken Access Control and Runtime Crash in /api/applications

Severity: High

CVSS Score: 7.5

Location: routes/api.py, get_applications(), line 141

Status: Open

Description: The company branch of the /api/applications endpoint references variable student which is never defined in that scope. The variable is defined only in the student branch above but Python's scoping rules make it accessible as an unbound local when the company branch executes. Additionally, drive_ids is computed but never used in the filter query, meaning the intended access control, restricting companies to only their own drive applications, was never functional.

Reproduction Steps:

1. Register and log in as a company account
2. Navigate to `http://127.0.0.1:5000/api/applications`
3. Observe `UnboundLocalError 500` response

Evidence:

UnboundLocalError

```
UnboundLocalError: cannot access local variable 'student' where it is not associated with a value
```

```
Traceback (most recent call last):

File "C:\Users\saini\Documents\PlacementPortal\env\Lib\site-packages\flaskapp.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\saini\Documents\PlacementPortal\env\Lib\site-packages\flaskapp.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\saini\Documents\PlacementPortal\env\Lib\site-packages\flaskapp.py", line 1511, in wsgi_app
    ctx = self.request_context(environ)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
error: BaseException | None = None
try:
    try:
        ctx.push()
        response = self.full_dispatch_request()
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    except Exception as e:
        error = e
        response = self.handle_exception(e)
    except:
        error = sys.exc_info()[1]
```

Vulnerable Code:

```
elif current_user.role == 'company':
```

```
company = CompanyProfile.query.filter_by(user_id=current_user.id).first()
```

```
drives = PlacementDrive.query.filter_by(company_id=current_user.id).all()
```

```
drive_ids = [d.id for d in drives]

applications = Application.query.filter_by(student_id=student.id).all()

# 'student' undefined, 'drive_ids' never used
```

Impact: The endpoint is entirely non-functional for company users. The intended access control logic, filtering applications to only those belonging to the company's drives was never implemented, meaning a corrected version without proper filtering could expose all applications across all companies.

Remediation:

```
elif current_user.role == 'company':

    company = CompanyProfile.query.filter_by(user_id=current_user.id).first()

    if not company:

        return api_not_found('Company profile not found')

    drives = PlacementDrive.query.filter_by(company_id=company.id).all()

    drive_ids = [d.id for d in drives]

    applications = Application.query.filter(

        Application.drive_id.in_(drive_ids)

    ).all()
```

VULN-002 Werkzeug Interactive Debugger Exposed

Severity: Critical

CVSS Score: 9.8

Location: app.py, app.run(debug=True)

Status: Open

Description: Flask is running with debug=True hardcoded in app.py. When any unhandled exception occurs, Werkzeug's interactive debugger is served directly in the browser response. The debugger provides a full Python REPL within each stack frame, allowing arbitrary code execution in the server process. This is triggered directly by VULN-001 — any company user navigating to /api/applications receives the interactive debugger.

Reproduction Steps:

1. Log in as any company account

2. Navigate to `http://127.0.0.1:5000/api/applications`
3. Observe Werkzeug debugger page with interactive console
4. Note: "You can execute arbitrary Python code in the stack frames"

Evidence:

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
```

```
File "C:\Users\saini\Documents\PlacementPortal\venv\Lib\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\saini\Documents\PlacementPortal\venv\Lib\site-packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\saini\Documents\PlacementPortal\venv\Lib\site-packages\flask\app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)  # type: ignore[no-any-return]
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\saini\Documents\PlacementPortal\venv\Lib\site-packages\flask_login\utils.py", line 290, in decorated_view
    return current_app.ensure_sync(func)(*args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\saini\Documents\PlacementPortal\routes\api.py", line 141, in get_applications
    applications = Application.query.filter_by(student_id = student_id).all()
    ^^^^^^^^^
UnboundLocalError: cannot access local variable 'student' where it is not associated with a value
```

The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error.

To switch between the interactive traceback and the plaintext one, you can click on the "Traceback" headline. From the text traceback you can also create a paste of it. For code execution mouse-over the frame you want to debug and click on the console icon on the right side.

You can execute arbitrary Python code in the stack frames and there are some extra helpers available for introspection:

- `dump()` shows all variables in the frame
- `dump(obj)` dumps all that's known about the object

Impact: Any authenticated user who triggers a server error gains the ability to execute arbitrary Python code on the server. This includes reading/writing files, accessing the database directly, creating new admin accounts, or executing system commands. Full system compromise is possible from a standard company account.

Remediation:

```
# app.py never hardcode debug=True
```

```
app.run(debug=os.environ.get('FLASK_DEBUG', 'False') == 'True')
```

```
# config.py add explicit production guard
```

```
if os.environ.get('FLASK_ENV') == 'production':
```

DEBUG = False

VULN-003 : Missing Explicit Role Enforcement on Resume Endpoint

Severity: Low

CVSS Score: 3.1

Location: routes/student.py, serve_resume()

Status: Partially Mitigated

Description: The /student/resume/<filename> endpoint lacks @role_required('student'). Access by non-student roles is currently prevented accidentally — get_student_profile() calls first_or_404() which returns 404 when no StudentProfile exists for a company or admin user ID. Student-to-student IDOR is correctly and intentionally prevented. The protection is functional but fragile and undocumented.

Evidence:

```
127.0.0.1 - - [02/May/2026 12:11:37] "GET /company/dashboard HTTP/1.1" 200 -  
DEBUG: role=company, filename=resume_3_1777703094.pdf  
127.0.0.1 - - [02/May/2026 12:11:50] "GET /student/resume/resume_3_1777703094.pdf HTTP/1.1" 404 -
```

Impact: Currently mitigated. Any future feature enabling cross-role profile lookup would silently remove this protection without any obvious security regression. The missing decorator is a latent vulnerability.

Remediation: Add @role_required('student') explicitly to the route decorator chain.

VULN-004 : HR Contact PII in Unauthenticated Public API Response

Severity: Low

CVSS Score: 3.7

Location: routes/api.py, get_drives()

Status: Open

Description: The /api/drives endpoint intentionally returns drive listings without authentication, consistent with public job board design. However the response includes hr_contact, which is not standard practice for unauthenticated job board APIs. Confirmed via direct browser access while logged out returning HTTP 200 with full HR contact data.

Evidence:

```
{  
  "drives": [  
    {  
      "company": {  
        "hr_contact": "9865320119",  
        "name": "asha",  
        "website": "https://samplecompany.org"  
      },  
      "deadline": "2026-05-04",  
      "eligibility_criteria": "Prior knowledge in VAPT, Ethical Hacking and Computer Networks",  
      "id": 1,  
      "job_description": "Log Analysis | VAPT Assistance | SIEM",  
      "job_title": "Cyber Security Intern"  
    },  
  ],  
  "total": 1  
}
```

Impact: HR contact information harvestable without any account. Enables spam, phishing, and social engineering against HR personnel at scale if the application is publicly hosted.

Remediation: Remove hr_contact from the unauthenticated response. Return it only after authentication:

```
'company': {  
    'name': d.company.company_name,  
    # hr_contact removed from public response  
    'website': d.company.website  
}
```

VULN-005: No Rate Limiting on Authentication Endpoint

Severity: Medium

CVSS Score: 5.3

Location: routes/auth.py, login()

Status: Open

Description: The /login endpoint accepts unlimited authentication attempts with no rate limiting, CAPTCHA, or account lockout. Testing confirmed 8 consecutive failed login attempts were accepted at full server speed with consistent 302 responses and no throttling.

Evidence:

```
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:42:29] "POST /login HTTP/1.1" 400 -  
127.0.0.1 - - [02/May/2026 13:44:08] "GET /logout HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:44:12] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:44:18] "POST /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:44:29] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:44:29] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:44:39] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:44:39] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:44:48] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:44:48] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:44:57] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:44:57] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:45:05] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:45:05] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:45:13] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:45:13] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:45:20] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:45:20] "GET /login HTTP/1.1" 200 -  
127.0.0.1 - - [02/May/2026 13:45:26] "POST /login HTTP/1.1" 302 -  
127.0.0.1 - - [02/May/2026 13:45:26] "GET /login HTTP/1.1" 200 -
```

Note: CSRF protection correctly blocked raw automated POST requests without valid tokens, confirming CSRF implementation is functional. However, this does not prevent browser-based or token-aware brute force attacks.

Impact: Unlimited password guessing against any known email address. Werkzeug's bcrypt hashing slows individual attempt verification but provides no lockout protection. A targeted attack against a known admin email is viable.

Remediation:

```
# Install flask-limiter

from flask_limiter import Limiter

limiter = Limiter(app, key_func=get_remote_address)

@auth.route('/login', methods=['GET', 'POST'])
@limiter.limit("5 per minute")
def login():
```

VULN-006 Hardcoded Secret Key Fallback Enables Session Forgery

Severity: Medium

CVSS Score: 5.9

Location: config.py

Status: Open

Description: SECRET_KEY falls back to the hardcoded string 'dev-secret-change-in-production' if the environment variable is not set. Flask uses this key to cryptographically sign session cookies. The key is publicly visible in the repository.

Vulnerable Code:

```
SECRET_KEY = os.environ.get('SECRET_KEY') or 'dev-secret-change-in-production'
```

Evidence:

```
import os

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or 'dev-secret-change-in-production'
```

Impact: Anyone with repository access can forge valid session cookies for any user ID including admin, achieving full privilege escalation without credentials. Risk is realised in any deployment where the environment variable is not explicitly configured.

Remediation:

Raise hard error if not configured, no silent fallback

```
SECRET_KEY = os.environ.get('SECRET_KEY')
```

if not SECRET_KEY:

```
    raise RuntimeError('SECRET_KEY environment variable is not set')
```

VULN-007 User Enumeration via Registration Form

Severity: Informational

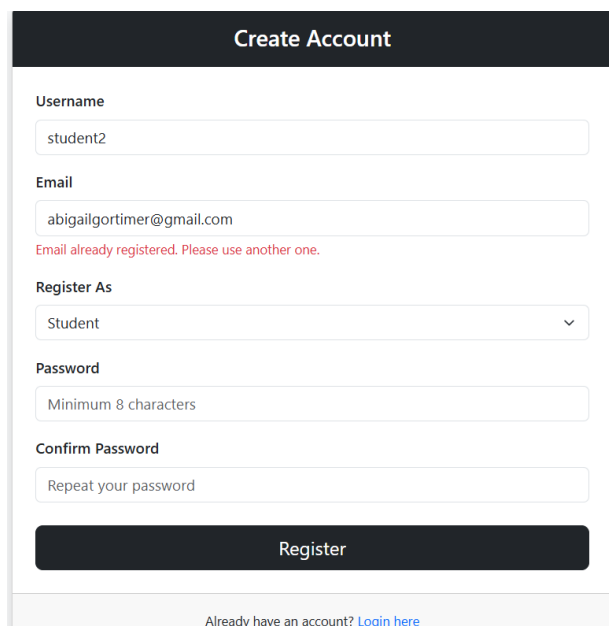
CVSS Score: 0.0

Location: forms.py, validate_username(), validate_email()

Status: Open

Description: Registration form returns distinct messages confirming whether a specific username or email already exists — "Username already taken" and "Email already registered." This allows an attacker to enumerate valid accounts.

Evidence:



The screenshot shows a 'Create Account' form with the following fields and values:

- Username:** student2
- Email:** abigailgortimer@gmail.com
- Register As:** Student (dropdown menu)
- Password:** Minimum 8 characters
- Confirm Password:** Repeat your password

A red error message is displayed below the email field: "Email already registered. Please use another one."

At the bottom of the form is a black 'Register' button. Below the form, there is a link: "Already have an account? [Login here](#)".

Impact: Enables confirmation of whether specific email addresses have accounts, feeding into targeted credential stuffing or phishing campaigns.

Remediation: Replace with a generic message: "Registration unsuccessful. Please check your details and try again."

Section 4 Remediation Summary Table

ID	Title	Severity	Status	Priority
VULN-002	Werkzeug debugger exposed	Critical	Open	Immediate
VULN-001	Broken access control in /api/applications	High	Open	Immediate
VULN-005	No rate limiting on login	Medium	Open	High
VULN-006	Hardcoded secret key fallback	Medium	Open	High
VULN-004	HR contact in public API	Low	Open	Medium
VULN-003	Missing explicit role check on resume	Low	Partially mitigated	Medium
VULN-007	User enumeration via registration	Informational	Open	Low

Section 5: Positive Security Observations

The following security controls were tested and found to be functioning correctly:

CSRF Protection: All automated POST requests without valid tokens were rejected with 400 Bad Request. Token validation is enforced server-side.

File Upload Validation: MIME type checking via python-magic correctly validates actual file content beyond extension and Content-Type header. Upload bypass attempts were unsuccessful.

Password Hashing: Werkzeug's PBKDF2 hashing is correctly applied to all stored passwords. No plaintext passwords observed in database.

Student IDOR Prevention: Student-to-student resume access is explicitly prevented with role-aware ownership checking.

Account Deactivation: Blacklisted accounts are correctly prevented from logging in via is_active check in authentication flow.